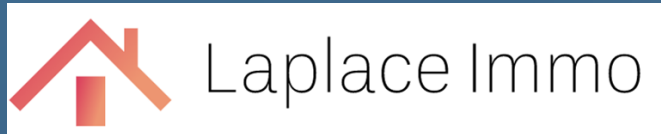


Analyse des ventes de biens immobiliers en 2020



Créer et Utiliser une
base de données
immobilières avec
SQL (et Python)

2 SOMMAIRE



I – PRESENTATION
DE L'ETUDE



II – LES DONNEES



III – LES ANALYSES

I-Présentation de l'étude

- A - LE CONTEXTE 
- B - LA METHODOLOGIE DE L'ANALYSE 
- C - LA STRATEGIE DE SAUVEGARDE et de CONFORMITE au RGPD 



A - LE CONTEXTE



A – LE CONTEXTE

- **Destinataire** : Laplace Immo, un **réseau national d'agences immobilières**
- **Objectif** : Créer une **base de données** collectant les **transactions immobilières**
- **But** : **analyser le marché** pour les différentes agences régionales



B - Méthodologie de l'analyse



B - Méthodologie de l'analyse I/2

- Etude et Nettoyage des données avec Python sous Jupyter Notebook :
 - Analyse des **attributs** et **type** de données
 - **Suppression des données inutiles, ou non conforme au RGPD**
 - **Nettoyage des données** (données vides, blanches, valeurs aberrantes)
 - **Retypage des données et noms de colonnes**
 - **Exportation en fichiers .CSV des tables nettoyées**
- Création du **Schéma relationnel** et **Script SQL** de Création de BDD sous **Looping**
- Base de données : PostgreSQL :
 - création des tables
 - importation des **Tables nettoyées** issues des fichiers CSV
 - **Requêtage SQL**



PostgreSQL



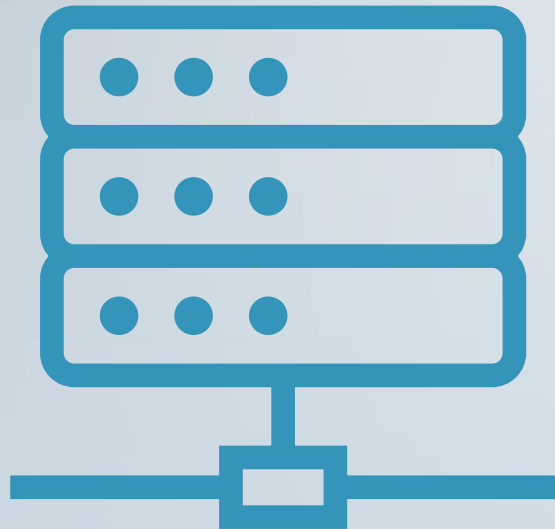
Laplace Immo

B - Méthodologie de l'analyse 2/2

- **Choix des Surfaces selon les usages immobiliers :**
 - réelle pour les maisons
 - Carrez pour les appartements (Surface applicable pour les biens en copropriété)



9 C - STRATEGIE DE SAUVEGARDE et CONFORMITE au RGPD



C - STRATEGIE DE SAUVEGARDE et de CONFOMRITE au RGPD

CONFORMITE AU RGPD :

- **Nettoyage à la racine (avec Python) :**
 - **Retrait des données personnelles** (Nom de l'acquéreur) et **des données inutiles**
 - **Fichier .csv** créés uniquement avec les **données utiles et conforme au RGPD**
- **SGBD n'intégrant que des données respectant le RGPD.**

STRATEGIE DE SAUVEGARDE :

- **Fichiers .csv :**
 - conforme au RGPD
 - intègres et **non modifié par la suite**
 - reproductibles à partir des fichiers sources (via Jupyter Notebook)**

II-LES DONNEES



- A - LES DONNEES INITIALES
- B - LE DICTIONNAIRE DES DONNEES
- C - LE SCHEMA RELATIONNEL NORMALISE
- D - LE NETTOYAGE DES DONNEES
- E - L'IMPLANTATIONS DANS LE SGBD des TABLES et des DONNEES



A - LES DONNÉES INITIALES



Fichier fr-es-referentiel-géographique.xlsx :

```
referentiel_geographique.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 38916 entries, 0 to 38915  
Data columns (total 37 columns):  
#   Column                Non-Null Count  Dtype    
---  ---                  
0   regrgp_nom            38916 non-null object   
1   reg_nom               38916 non-null object   
2   reg_nom_old           38916 non-null object   
3   aca_nom               38916 non-null object   
4   dep_nom               38916 non-null object   
5   com_code              38916 non-null object   
6   com_code1             38916 non-null object   
7   com_code2             38916 non-null object   
8   com_id                38916 non-null object   
9   com_nom_maj_court     38916 non-null object   
10  com_nom_maj           38916 non-null object   
11  com_nom               38916 non-null object   
12  uu_code               7625 non-null  object   
13  uu_id                 38916 non-null object   
14  uucr_id               38916 non-null object   
15  uucr_nom              38916 non-null object
```

```
16  ze_id                38916 non-null object   
17  dep_code              38916 non-null object   
18  dep_id               38916 non-null object   
19  dep_nom_num          38916 non-null object   
20  dep_num_nom          38916 non-null object   
21  aca_code              38916 non-null int64    
22  aca_id               38916 non-null object   
23  reg_code              38916 non-null int64    
24  reg_id               38916 non-null object   
25  reg_code_old          38916 non-null int64    
26  reg_id_old            38916 non-null object   
27  fd_id                38916 non-null object   
28  fr_id                38916 non-null object   
29  fe_id                38916 non-null object   
30  uu_id_99             38916 non-null object   
31  au_code              21444 non-null object   
32  au_id                38916 non-null object   
33  auc_id               38916 non-null object   
34  auc_nom              38916 non-null object   
35  uu_id_10             38916 non-null object   
36  geolocalisation      36745 non-null object   
dtypes: int64(3), object(34)  
memory usage: 11.0+ MB
```



Fichier donnees_communes.xlsx :

```
donnees_communes.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 34991 entries, 0 to 34990  
Data columns (total 9 columns):  
#   Column      Non-Null Count  Dtype  
---  -  
0   CODREG      34991 non-null  int64  
1   CODDEP      34991 non-null  object  
2   CODARR      34990 non-null  float64  
3   CODCAN      34990 non-null  object  
4   CODCOM      34991 non-null  int64  
5   COM         34991 non-null  object  
6   PMUN        34991 non-null  int64  
7   PCAP        34991 non-null  int64  
8   PTOT        34991 non-null  int64  
dtypes: float64(1), int64(5), object(3)  
memory usage: 2.4+ MB
```



Fichier valeurs_foncières.xlsx :

```
valeurs_foncières.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 34169 entries, 0 to 34168
```

```
Data columns (total 46 columns):
```

#	Column	Non-Null Count	Dtype
0	Code service CH	0 non-null	float64
1	Reference document	0 non-null	float64
2	1 Articles CGI	0 non-null	float64
3	2 Articles CGI	0 non-null	float64
4	3 Articles CGI	0 non-null	float64
5	4 Articles CGI	0 non-null	float64
6	5 Articles CGI	0 non-null	float64
7	No disposition	34169 non-null	int64
8	Date mutation	34169 non-null	datetime64[ns]
9	Nature mutation	34169 non-null	object
10	Valeur foncière	34151 non-null	float64
11	No voie	34036 non-null	float64
12	B/T/Q	2174 non-null	object
13	Code type de voie	34169 non-null	int64
14	Type de voie	33229 non-null	object
15	Code voie	34169 non-null	object
16	Voie	34169 non-null	object
17	Code ID commune	34169 non-null	int64
18	Code postal	34168 non-null	float64
19	Commune	34169 non-null	object

20	Code département	34169 non-null	object
21	Code commune	34169 non-null	int64
22	Préfixe de section	1143 non-null	float64
23	Section	34168 non-null	object
24	No plan	34169 non-null	int64
25	No Volume	0 non-null	float64
26	1er lot	34169 non-null	object
27	Surface Carrez du 1er lot	34169 non-null	float64
28	2eme lot	0 non-null	float64
29	Surface Carrez du 2eme lot	0 non-null	float64
30	3eme lot	0 non-null	float64
31	Surface Carrez du 3eme lot	0 non-null	float64
32	4eme lot	0 non-null	float64
33	Surface Carrez du 4eme lot	0 non-null	float64
34	5eme lot	0 non-null	float64
35	Surface Carrez du 5eme lot	0 non-null	float64
36	Nombre de lots	34169 non-null	int64
37	Code type local	34169 non-null	int64
38	Type local	34169 non-null	object
39	Identifiant local	0 non-null	float64
40	Surface réelle bâti	34169 non-null	int64
41	Nombre pièces principales	34169 non-null	int64
42	Nature culture	253 non-null	object
43	Nature culture spéciale	8 non-null	object
44	Surface terrain	253 non-null	float64
45	Nom de l'acquéreur	34169 non-null	object

dtypes: datetime64[ns](1), float64(23), int64(9), object(13)
memory usage: 12.0+ MB



B - LE DICTIONNAIRE DES DONNEES



Le dictionnaire des données

DICTIONNAIRE DES DONNÉES - Référentiel géographique

CODE ▼	CODE source ▼	SIGNIFICATION ▼	A CONSERVER ▼	TYPE ▼	LONGUEU ▼	NATURE ▼	REGLE DE GESTION ▼	REGLE DE CALCUL ▼	champ de valeur ▼
Nom_region	reg_nom	Libellé région	UTILE / A CONSERVER	VARCHAR	100	Elémentaire	NOT NULL		
Nom_departement	dep_nom	Libellé du département	UTILE / A CONSERVER	VARCHAR	100	Elémentaire	NOT NULL		
Code_departement	dep_code	Code département	UTILE / A CONSERVER	VARCHAR	3	Elémentaire	NOT NULL		
Id_departement	dep_id	Id du département	UTILE / A CONSERVER	CHAR	4	Elémentaire	NOT NULL	dep_id (4carac)=D+0 ou 00 si nec+code dep(1 à 3 carac)	dep_id (4carac)=D+0 ou 00 si nec+code dep(1 à 3 carac)
Code_region	reg_code	Code région	UTILE / A CONSERVER	VARCHAR	2	Elémentaire	NOT NULL		1 à 2 caractères numériques (de 0 à 94)
Id_region	reg_id	Id région	UTILE / A CONSERVER	CHAR	3	Elémentaire	NOT NULL	reg_id(3carac)=R+0si nec+code reg(1 à 2 car)	reg_id(3carac)=R+0si nec+code reg(1 à 2 car)

mis dans Looping pour Commune

mis dans Looping pour Departement

mis dans Looping pour Region



Laplace Immo

Le dictionnaire des données

DICTIONNAIRE DES DONNÉES - Données communes

CODE	CODE Sou	SIGNIFICATION	A CONSERVER	TYPE	LONGUEU	NATURE	REGLE DE GESTION	REGLE DE CALCUL	champ de valeur
Id_codedep_codecommune	/ com_id ds réf géo	Identifiant de la commu	UTILE / A CRÉER	CHAR	6	Elémentaire	NON VIDE	id_codedep_codecomm=C+ 2 premiers caractères de CODDEP (tronquer 974 en 97) +les 3 caractères de CODCOM	id_codedep_codecomm=C+ 2 premiers caractères de CODDEP (tronquer 974 en 97) +les 3 caractères de CODCOM
Code_commune	CODCOM	Code Commune	UTILE / A CONSERVER	CHAR	3	Elémentaire	NON VIDE	codcom=3 derniers caractères de idcodedep_codecommune	valeurs de 001 à 909
Nom_commune	COM	Nom de commune	UTILE / A CONSERVER	VARCHAR	100	Elémentaire			au min 38 caractères ex : Escueillens- et-Saint-Just-de-Bélengard
Population_totale	PTOT	Population totale	UTILE / A CONSERVER	INT	INT32	Elémentaire		PTOT=PMUN+PCAP	velures entière inf à 500 000
Id_departement	/dep_id ds réf geo	Id du département	UTILE / A CRÉER Clé étrangère	CHAR	4	Elémentaire	NON VIDE	dep_id (4carac)=D+0 ou 00 si nec+code dep(1 à 3 carac)	dep_id (4carac)=D+0 ou 00 si nec+code dep(1 à 3 carac)

mis dans Looping pour Commune



Le dictionnaire des données

DICTIONNAIRE DES DONNÉES - Valeurs foncières

CODE	CODE source	SIGNIFICATION	A CONSERVER	TYPE	LONGUEU	NATURE	REGLE DE GESTION	REGLE DE CALCUL	champ de valeur
Id_vente		ID vente dans la base de données	UTILE / A CRÉER	VARCHAR	10	Elémentaire	Ne doit pas être nul		
Id_bien		ID dans la base de données	UTILE / A CRÉER	VARCHAR	10	Elémentaire	Ne doit pas être nul		
No_voie	No voie	Numéro des rues	UTILE / A CONSERVER	SMALLINT (INT16)	INT16	Elémentaire			
Type_de_voie	Type de voie	Plusieurs valeurs (rue, avenue, chemin, etc.)	UTILE / A CONSERVER	VARCHAR	10	Elémentaire			
Voie	Voie	Nom de la rue	UTILE / A CONSERVER	VARCHAR	60	Elémentaire			
Date_vente	Date mutation	Date de signature de l'acte	UTILE / A CONSERVER	DATE		Elémentaire			
Valeur	Valeur fonciere	prix net vendeur (ou évaluation)	UTILE / A CONSERVER	DECIMAL(11,2)	DECIMAL(11,2)	Elémentaire			Float jusqu'à 9Millions
BTQ	B/T/Q	Indice de répétition	UTILE / A CONSERVER	CHAR	1	Elémentaire	peut être vide		
Code_postal	Code postal		UTILE / A CONSERVER	INT	INT32	Elémentaire			4 à 5 carac num
Surface_carrez	Surface Carrez du 1er lot	plafond	UTILE / A CONSERVER	DECIMAL(7,2)	DECIMAL(7,2)	Elémentaire	NON NULL/NON VIDE		float entre 0 et 5000
Type_local	Type local	3 : dépendance (isolée) ; 4 :	UTILE / A CONSERVER	VARCHAR	20	Elémentaire			alphan (maison ou appartement)
Surface_reelle	Surface reelle bati	mesurée au sol entre les murs	UTILE / A CONSERVER	SMALLINT (INT16)	INT16	Elémentaire	NON VIDE		entier (de 0 à 500)
Total_piece	Nombre pieces principales		UTILE / A CONSERVER	SMALLINT (INT16)	INT16	Elémentaire			entier (val de 0 à 11)
Surface_terrain	Surface terrain		UTILE / A CONSERVER	SMALLINT (INT16)	INT16	Elémentaire	peut être vide		entier (val de 1 à 2670
Id_codedep_codecommune		voir com_id ds ref geo	UTILE / A CRÉER	CHAR	6	Elémentaire		Id_codedep_codecomm = C+CODEP(restreint aux 2 premiers car)+CODECOM(3car)	6 caractères alphanum de la form CXXXXXX C2AXXX possib C974XX possib

mis dans MCD Looping pour Bien
 mis dans MCD Looping pour Vente
 mis dans Looping pour Commune

C - LE SCHEMA RELATIONNEL NORMALISE

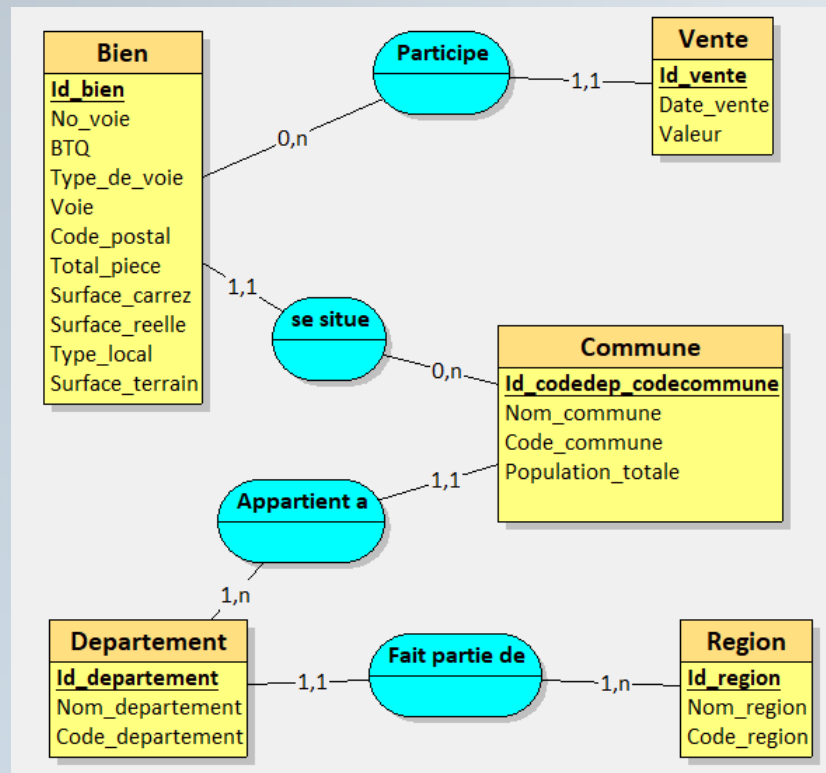


Looping

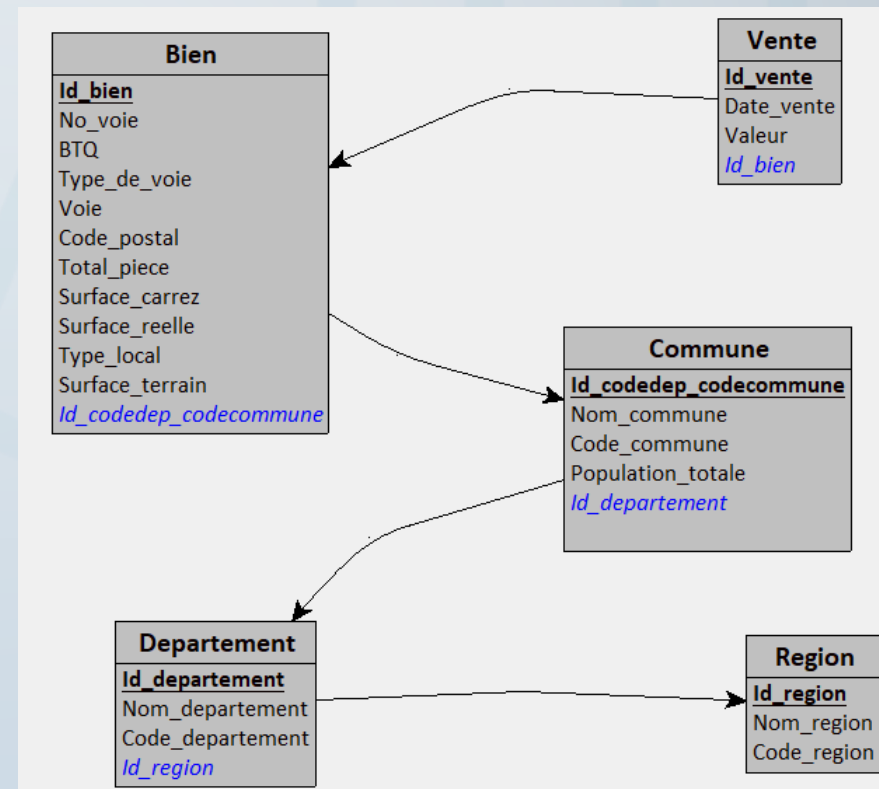


C - LE SCHEMA RELATIONNEL NORMALISE (3NF)

MCD(MERISE)



MLD



D - LE NETTOYAGE DES DONNEES



D - Le nettoyage des données :



- Suppression de colonnes inutiles et/ou contenant des données personnelles
- Analyse de valeurs manquantes
- Analyse de valeurs aberrantes
- Création de colonnes utiles
- Retypage de colonnes



Fichiers Region et Departement à partir du fichier referentiel_geographique



```
Region_final.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 19 entries, 0 to 18  
Data columns (total 3 columns):  
#   Column      Non-Null Count  Dtype  
---  ---  
0   Id_region    19 non-null    object  
1   Nom_region   19 non-null    object  
2   Code_region  19 non-null    int64  
dtypes: int64(1), object(2)  
memory usage: 588.0+ bytes
```

```
Departement_final.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 109 entries, 0 to 108  
Data columns (total 4 columns):  
#   Column      Non-Null Count  Dtype  
---  ---  
0   Id_departement  109 non-null    object  
1   Nom_departement  109 non-null    object  
2   Code_departement  109 non-null    object  
3   Id_region        109 non-null    object  
dtypes: object(4)  
memory usage: 3.5+ KB
```


Fichier Commune à partir du fichier donnees_communes



```
Commune_final.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 34991 entries, 0 to 34990
```

```
Data columns (total 5 columns):
```

#	Column	Non-Null Count	Dtype
0	Id_codedep_codecommune	34991 non-null	object
1	Nom_commune	34991 non-null	object
2	Code_commune	34991 non-null	object
3	Population_totale	34991 non-null	int64
4	Id_departement	34991 non-null	object

```
dtypes: int64(1), object(4)
```

```
memory usage: 1.3+ MB
```



Valeurs_foncières avant Nettoyage pour les tables Bien et Vente



Description_Fichier_Tableau_avc_ValBlanches(valeurs_foncières_nettoyé_1[liste_col_utiles_VF])

	Type	Nb lignes	Valeurs non-vides	Valeurs vides	Valeurs distinctes	Valeurs blanches
No voie	float64	34169	34036	133	1469	0
Type de voie	object	34169	33229	940	80	0
Voie	object	34169	34169	0	14133	0
B/T/Q	object	34169	2174	31995	25	0
Code postal	float64	34169	34168	1	2450	0
Surface Carrez du 1er lot	float64	34169	34169	0	10398	0
Type local	object	34169	34169	0	2	0
Surface réelle bati	int64	34169	34169	0	256	0
Nombre pièces principales	int64	34169	34169	0	12	0
Surface terrain	float64	34169	253	33916	201	0
Code département	object	34169	34169	0	96	0
Code commune	int64	34169	34169	0	666	0
Date mutation	datetime64[ns]	34169	34169	0	158	0
Valeur foncière	float64	34169	34151	18	9681	0



Nettoyage sur le fichier Valeurs_foncières :

(détail en [annexe](#))



- **34 169 lignes** dans le fichier d'origine valeurs_foncières.xlsx
- **34 169 lignes** en entrée dans le dataframe Jupyter Notebook avant Nettoyage

NETTOYAGES et retrait de valeurs aberrantes laissant penser à des erreurs:

- AJOUT 1 code postal : pas de chgmt nbr de lignes
- Remplissage de valeur vides (N° Voie et Surface Terrain à 0, Type de voie et B/T/Q à "") : pas de chgmt nbr de lignes
- **SUPPRESSIONS de 18 lignes** qui n'ont **pas de valeurs foncières** (NAN) : => 34 151 lignes
- **SUPPRESSION de 2 lignes** avec des **surfaces réelles bâties non habitables** <= 4m2 => 34 149 lignes
- **SUPPRESSION de 3 lignes** avec des **valeur foncières incohérentes(erronées)** <= 2_500 euros : => 34 146 lignes
- Etude des doublons de Biens => Pas de suppressions, Pas d'impact sur l'étude.

FICHIERS POST NETTOYAGE :

- **Fichiers .CSV** Bien.csv et Vente.csv : **34 146 lignes**
- Dans ma **base de données** pour les Tables Bien et Vente : **34 146 lignes**

Fichier valeur foncières après nettoyage et retypepage



Description_Fichier_Tableau_avc_ValBlanches(valeurs_foncières_NETTOYE_RETYPE_FINAL[liste_col_utiles_VF])

	Type	Nb lignes	Valeurs non-vides	Valeurs vides	Valeurs distinctes	Valeurs blanches
No voie	int16	34146	34146	0	1468	0
Type de voie	object	34146	34146	0	80	940
Voie	object	34146	34146	0	14130	0
B/T/Q	object	34146	34146	0	25	31973
Code postal	int32	34146	34146	0	2449	0
Surface Carrez du 1er lot	float64	34146	34146	0	10394	0
Type local	object	34146	34146	0	2	0
Surface réelle bati	int64	34146	34146	0	254	0
Nombre pièces principales	int64	34146	34146	0	12	0
Surface terrain	int16	34146	34146	0	201	0
Code département	object	34146	34146	0	96	0
Code commune	int64	34146	34146	0	666	0
Date mutation	datetime64[ns]	34146	34146	0	158	0
Valeur foncière	float64	34146	34146	0	9677	0



FICHER Bien et Fichier Vente



```
Bien_final.info()
```

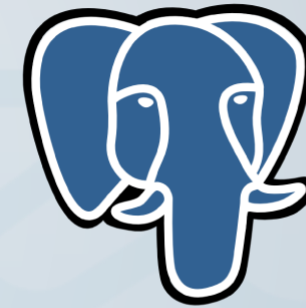
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 34146 entries, 0 to 34145
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Id_bien                34146 non-null  string
1   No_voie                34146 non-null  int16
2   BTQ                    34146 non-null  object
3   Type_de_voie           34146 non-null  object
4   Voie                   34146 non-null  object
5   Code_postal            34146 non-null  int32
6   Total_piece            34146 non-null  int64
7   Surface_carrez         34146 non-null  float64
8   Surface_reelle         34146 non-null  int64
9   Type_local             34146 non-null  object
10  Surface_terrain        34146 non-null  int16
11  Id_codedep_codecommune 34146 non-null  object
dtypes: float64(1), int16(2), int32(1), int64(2), object(5), string(1)
memory usage: 2.6+ MB
```

```
Vente_final.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 34146 entries, 0 to 34145
Data columns (total 4 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Id_vente              34146 non-null  string
1   Date_vente            34146 non-null  datetime64[ns]
2   Valeur                34146 non-null  float64
3   Id_bien               34146 non-null  string
dtypes: datetime64[ns](1), float64(1), string(2)
memory usage: 1.0 MB
```



E - L'IMPLANTATIONS DANS LE SGBD des TABLES et des DONNEES



PostgreSQL



Laplace Immo

Le SCRIPT SQL de création des tables :



```
CREATE TABLE Region(  
  Id_region CHAR(3),  
  Nom_region VARCHAR(100) NOT NULL,  
  Code_region VARCHAR(2),  
  PRIMARY KEY(Id_region)  
);
```

```
CREATE TABLE Departement(  
  Id_departement CHAR(4),  
  Nom_departement VARCHAR(100) NOT NULL,  
  Code_departement VARCHAR(3),  
  Id_region CHAR(3) NOT NULL,  
  PRIMARY KEY(Id_departement),  
  FOREIGN KEY(Id_region) REFERENCES Region(Id_region)  
);
```

```
CREATE TABLE Commune(  
  Id_codedep_codecommune CHAR(6),  
  Nom_commune VARCHAR(100) NOT NULL,  
  Code_commune CHAR(3),  
  Population_totale INT,  
  Id_departement CHAR(4) NOT NULL,  
  PRIMARY KEY(Id_codedep_codecommune),  
  FOREIGN KEY(Id_departement) REFERENCES Departement(Id_departement)  
);
```

```
CREATE TABLE Bien(  
  Id_bien VARCHAR(10),  
  No_voie SMALLINT,  
  BTQ CHAR(1),  
  Type_de_voie VARCHAR(10),  
  Voie VARCHAR(60) NOT NULL,  
  Code_postal INT,  
  Total_piece SMALLINT,  
  Surface_carrez DECIMAL(7,2),  
  Surface_reelle SMALLINT,  
  Type_local VARCHAR(20) NOT NULL,  
  Surface_terrain SMALLINT,  
  Id_codedep_codecommune CHAR(6) NOT NULL,  
  PRIMARY KEY(Id_bien),  
  FOREIGN KEY(Id_codedep_codecommune) REFERENCES Commune(Id_codedep_codecommune)  
);
```

```
CREATE TABLE Vente(  
  Id_vente VARCHAR(10),  
  Date_vente DATE NOT NULL,  
  Valeur DECIMAL(11,2),  
  Id_bien VARCHAR(10) NOT NULL,  
  PRIMARY KEY(Id_vente),  
  FOREIGN KEY(Id_bien) REFERENCES Bien(Id_bien)  
);
```



Tables Region et Departement



10
11 **SELECT * FROM Region ;**
12
13 **SELECT * FROM Departement ;**

Data Output Messages Notifications

	id_region [PK] character (3)	nom_region character varying (100)	cod cha
1	R00	Collectivités d'outre-mer	0
2	R01	Guadeloupe	1
3	R02	Martinique	2
4	R03	Guyane	3
5	R04	La Réunion	4
6	R06	Mayotte	6
7	R11	Ile-de-France	11
8	R24	Centre-Val de Loire	24
9	R27	Bourgogne-Franche-Comté	27
10	R28	Normandie	28
11	R32	Hauts-de-France	32
12	R44	Grand Est	44
13	R52	Pays de la Loire	52
14	R53	Bretagne	53
15	R75	Nouvelle-Aquitaine	75
16	R76	Occitanie	76
17	R84	Auvergne-Rhône-Alpes	84
18	R93	Provence-Alpes-Côte d'Azur	93
19	R94	Corse	94

Total rows: 19 of 19 Query complete 00:00:00.316

12
13 **SELECT * FROM Departement ;**

Data Output Messages Notifications

	id_departement [PK] character (4)	nom_departement character varying (100)
1	D001	Ain
2	D002	Aisne
3	D003	Allier
4	D004	Alpes-de-Haute-Provence
5	D005	Hautes-Alpes
6	D006	Alpes-Maritimes
7	D007	Ardèche
8	D008	Ardennes
9	D009	Ariège
10	D010	Aube
11	D011	Aude
12	D012	Aveyron
13	D013	Bouches-du-Rhône
14	D014	Calvados
15	D015	Cantal
16	D016	Charente
17	D017	Charente-Maritime
18	D018	Cher
19	D019	Corrèze
20	D021	Côte-d'Or

Total rows: 109 of 109 Query complete 00:00:00.316



Table Commune

14
15 `SELECT * FROM Commune ;`

Data Output Messages Notifications

	<code>id_codedep_codecommune</code> [PK] character (6)	<code>nom_commune</code> character varying (100)	<code>code_commune</code> character (3)	<code>population_totale</code> integer	<code>id_departement</code> character (4)
1	C01001	L'Abergement-Clémenciat	001	798	D001
2	C01002	L'Abergement-de-Varey	002	257	D001
3	C01004	Ambérieu-en-Bugey	004	14514	D001
4	C01005	Ambérieux-en-Dombes	005	1776	D001
5	C01006	Ambléon	006	118	D001
6	C01007	Ambronay	007	2915	D001
7	C01008	Ambutrix	008	777	D001
8	C01009	Andert-et-Condou	009	335	D001
9	C01010	Anglefort	010	1122	D001
10	C01011	Apremont	011	379	D001
11	C01012	Aranc	012	333	D001
12	C01013	Arandas	013	147	D001
13	C01014	Arbent	014	3442	D001
14	C01015	Arbois en Bugey	015	667	D001
15	C01016	Arbigny	016	469	D001
16	C01017	Argis	017	473	D001
17	C01019	Armix	019	31	D001
18	C01021	Ars-sur-Formans	021	1485	D001
19	C01022	Artemare	022	1267	D001
20	C01023	Asnières-sur-Saône	023	76	D001
Total rows: 1000 of 34991 Query complete 00:00:00.419 Ln 15, Col 1					

Tables Bien et Vente



1.3 Sequences

Tables (5)

bien

Columns (12)

- id_bien
- no_voie
- btq
- type_de_voie
- voie
- code_postal
- total_piece
- surface_carrez
- surface_reelle
- type_local
- surface_terrain
- id_codedep_codecommune

Constraints

Indexes

RLS Policies

Rules

Triggers

commune

departement

region

vente

Trigger Functions

Tvdes

17

SELECT * FROM Bien ;

Data Output Messages Notifications

	id_bien [PK] character varying (10)	no_voie smallint	btq character (1)	type_de_voie character varying (1)
1	B0	347	[null]	RUE
2	B1	4	[null]	BD
3	B2	20	B	RUE
4	B3	550	[null]	RTE
5	B4	9300	[null]	RES
6	B5	27	[null]	RUE
7	B6	360	[null]	AV
8	B7	5076	F	PARC
9	B8	1194	[null]	RUE
10	B9	30	[null]	ALL
11	B10	11	[null]	RUE
12	B11	13	[null]	RUE
13	B12	1	[null]	RUE
14	B13	2	[null]	RUE
15	B14	5	[null]	AV
16	B15	15	[null]	RUE
17	B16	176	[null]	RUE
18	B17	822	G	[null]
19	B18	9	[null]	RUE

Total rows: 1000 of 34146 Query complete 00:00:00.421 Ln 17, Col 1

1.3 Sequences

Tables (5)

bien

commune

departement

region

vente

Columns (4)

- id_vente
- date_vente
- valeur
- id_bien

Constraints

Indexes

RLS Policies

Rules

Triggers

Trigger Functions

Types

Views (11)

Subscriptions

Test 1

Test_2_N24

Test_3_N24

postgres

bin/Group Roles

lespaces

18

19

SELECT * FROM Vente ;

Data Output Messages Notifications

	id_vente [PK] character varying (10)	date_vente date	valeur numeric (11,2)	id_bien character varying (10)
1	V0	2020-01-02	165000.00	B0
2	V1	2020-01-02	355680.00	B1
3	V2	2020-01-02	229500.00	B2
4	V3	2020-01-02	125000.00	B3
5	V4	2020-01-02	90000.00	B4
6	V5	2020-01-02	93000.00	B5
7	V6	2020-01-02	298100.00	B6
8	V7	2020-01-02	163500.00	B7
9	V8	2020-01-02	53000.00	B8
10	V9	2020-01-02	136000.00	B9
11	V10	2020-01-02	125900.00	B10
12	V11	2020-01-02	234000.00	B11
13	V12	2020-01-02	46210.00	B12
14	V13	2020-01-02	129000.00	B13
15	V14	2020-01-02	122500.00	B14
16	V15	2020-01-02	86025.00	B15
17	V16	2020-01-02	79000.00	B16
18	V17	2020-01-02	81795.00	B17
19	V18	2020-01-02	156000.00	B18
20	V19	2020-01-02	139900.00	B19

Total rows: 1000 of 34146 Query complete 00:00:00.487 Ln 19, Col 1

III – LES ANALYSES



- Le code SQL des REQUETES et LES RESULTATS



Les requêtes :

- 1. Nombre total d'appartements vendus au 1er semestre 2020.
- 2. Le nombre de ventes d'appartement par région pour le 1er semestre 2020.
- 3. Proportion des ventes d'appartements par le nombre de pièces.
- 4. Liste des 10 départements où le prix du mètre carré est le plus élevé.
- 5. Prix moyen du mètre carré d'une maison en Île-de-France.
- 6. Liste des 10 appartements les plus chers avec la région et le nombre de mètres carrés.
- 7. Taux d'évolution du nombre de ventes entre le premier et le second trimestre de 2020.
- 8. Le classement des régions par rapport au prix au mètre carré des appartement de plus de 4 pièces.
- 9. Liste des communes ayant eu au moins 50 ventes au 1er trimestre
- 10. Différence en pourcentage du prix au mètre carré entre un appartement de 2 pièces et un appartement de 3 pièces.
- 11. Les moyennes de valeurs foncières pour le top 3 des communes des départements 6, 13, 33, 59 et 69.
- 12. Les 20 communes avec le plus de transactions pour 1000 habitants pour les communes qui dépassent les 10 000 habitants.

0. Création d'une vue globale :VUE_GLOBALE :

--CREATION D'UNE VUE GLOBALE (env34000 lignes aussi raisonnable que Vente ou Bien)

```
CREATE VIEW VUE_GLOBALE AS
SELECT      V.Id_vente , V.Date_vente, V.Valeur,
            B.*,
            Co.Nom_commune, Co.Code_commune, Co.Population_totale,
            D.*,
            R.Nom_region, R.Code_region

FROM Vente V
INNER JOIN Bien B ON V.Id_bien = B.Id_bien
INNER JOIN Commune Co ON B.Id_codedep_codecommune =
Co.Id_codedep_codecommune
INNER JOIN Departement D ON Co.Id_departement = D.Id_departement
INNER JOIN Region R ON D.Id_region = R.Id_region
;

SELECT * FROM VUE_GLOBALE ;
```



The screenshot displays a database interface with a tree view on the left showing the 'vue_globale' view and its 24 columns. The columns listed are: id_vente, date_vente, valeur, id_bien, no_voie, btq, type_de_voie, voie, code_postal, total_piece, surface_carrez, surface_reelle, type_local, surface_terrain, id_codedep_codecommune, nom_commune, code_commune, population_totale, id_departement, nom_departement, code_departement, id_region, nom_region, and code_region. The main pane on the right shows the SQL query used to create the view and its output. The query is: `INNER JOIN Region R ON D.Id_region = R.Id_region ; SELECT * FROM VUE_GLOBALE ;`. The 'Data Output' tab shows a table with 17 rows and 4 columns: id_vente (character varying (10)), date_vente (date), valeur (numeric (11,2)), and id_bien (character varying (10)). The data shows a list of properties with their sale dates (all 2020-01-02) and values. At the bottom, it indicates 'Total rows: 1000 of 34146' and 'Query complete 00:00:00.584 Ln 19, Col 1'.

	id_vente	date_vente	valeur	id_bien
1	V0	2020-01-02	165000.00	B0
2	V1	2020-01-02	355680.00	B1
3	V2	2020-01-02	229500.00	B2
4	V3	2020-01-02	125000.00	B3
5	V4	2020-01-02	90000.00	B4
6	V5	2020-01-02	93000.00	B5
7	V6	2020-01-02	298100.00	B6
8	V7	2020-01-02	163500.00	B7
9	V8	2020-01-02	53000.00	B8
10	V9	2020-01-02	136000.00	B9
11	V10	2020-01-02	125900.00	B10
12	V11	2020-01-02	234000.00	B11
13	V12	2020-01-02	46210.00	B12
14	V13	2020-01-02	129000.00	B13
15	V14	2020-01-02	122500.00	B14
16	V15	2020-01-02	86025.00	B15
17	V16	2020-01-02	79000.00	B16



1. Nbre total d'appartements vendus au 1er semestre 2020.

--REQUETE 1 : Nombre total d'appartements vendus au 1er semestre 2020.

SELECT COUNT(Type_local) AS "Nombre d'appartement vendus au cours du 1er semestre : "

FROM VUE_GLOBALE
WHERE '01/01/2020' <= Date_Vente
AND Date_Vente < '01/07/2020'
AND Type_local = 'Appartement'
;



Data Output		Messages	Notifications
		         	
	Nombre d'appartement vendus au cours du 1er semestre : bigint		
1			31357



2. Nbre d'appartement vendus **par région** au 1er semestre 2020.

--REQUETE 2 : Le nombre de ventes d'appartement par région pour le 1er semestre 2020.

```
SELECT      Id_region,
            Nom_region AS Region,
            COUNT(DISTINCT Id_vente) AS "Nombre de vente
d'appartement au 1er semestre 2020"

FROM VUE_GLOBALE
WHERE '01/01/2020' <= Date_Vente
AND Date_Vente < '01/07/2020'
AND Type_local = 'Appartement'
GROUP BY Id_region, Nom_region
ORDER BY "Nombre de vente d'appartement au 1er semestre 2020"
DESC
;
```



Data Output Messages Notifications			
SQL			
	id_region character (3)	region character varying (100)	Nombre de vente d'appartement au 1er semestre 2020 bigint
1	R11	Ile-de-France	13988
2	R93	Provence-Alpes-Côte d'Azur	3645
3	R84	Auvergne-Rhône-Alpes	3252
4	R75	Nouvelle-Aquitaine	1931
5	R76	Occitanie	1640
6	R52	Pays de la Loire	1357
7	R32	Hauts-de-France	1252
8	R44	Grand Est	984
9	R53	Bretagne	982
10	R28	Normandie	861
11	R24	Centre-Val de Loire	693
12	R27	Bourgogne-Franche-Comté	376
13	R94	Corse	222
14	R02	Martinique	94
15	R04	La Réunion	44
16	R03	Guyane	34
17	R01	Guadeloupe	2
Total rows: 17 of 17 Query complete 00:00:00.495 Ln 45, Col 2			



3. Proportion des ventes d'appartements par le nombre de pièces.

```
-- PRE-REQUETE 3 ETUDE DU SELECT : Total_appart_vendu  
SELECT      COUNT(*) AS Total_appart_vendu  
FROM VUE_GLOBALE  
WHERE Type_local = 'Appartement'
```

	total_appart_vendu bigint
1	31357

```
SELECT      Total_piece AS "Nbr de pièces",  
            ROUND(      ROUND(COUNT(*)*100,2)  
                        / (SELECT      COUNT(*) AS Total_appart_vendu  
                          FROM VUE_GLOBALE  
                          WHERE Type_local = 'Appartement')  
                        ,2) AS "Pourcentage des appartements vendus",  
            COUNT(*)AS Total_appart_vendu  
  
FROM VUE_GLOBALE  
WHERE Type_local = 'Appartement'  
GROUP BY Total_piece  
ORDER BY Total_piece ASC;
```

	Nbr de pièces smallint	Pourcentage des appartements vendus numeric	total_appart_vendu bigint
1	0	0.09	29
2	1	21.48	6734
3	2	31.17	9773
4	3	28.59	8964
5	4	14.22	4458
6	5	3.55	1114
7	6	0.65	203
8	7	0.17	54
9	8	0.05	17
10	9	0.03	8
11	10	0.01	2
12	11	0.00	1
Total rows: 12 of 12 Query complete 00:00:00.189 Ln 70, Col 1			



4. Les 10 départements où le prix du mètre carré est le plus élevé

–Version 1 : chaque m2 à poids équivalent

Version 1:

Traitement avec somme total des m2 et des valeurs
Chaque mètre carré a un poids équivalent

```
SELECT      Nom_departement,
            SUM(Valeur) AS Total_Valeur,
            SUM(Surface_reelle) AS Total_Surface,
            SUM(Valeur)/SUM(Surface_reelle) AS Prix_m2

FROM VUE_GLOBALE
GROUP BY Nom_departement
ORDER BY Prix_m2 DESC
LIMIT 10;
```

	nom_departement character varying (100)	total_valeur numeric	total_surface bigint	prix_m2 numeric
1	Paris	2973890357.39	248158	11983.858498980488
2	Hauts-de-Seine	744006162.02	101100	7359.1113948565776459
3	Val-de-Marne	535989451.38	106415	5036.7847707560024433
4	Alpes-Maritimes	331274235.02	71836	4611.5350941032351467
5	Seine-Saint-Denis	230731274.69	55852	4131.1192918785361312
6	Haute-Savoie	81635375.89	20176	4046.1625639373513085
7	Yvelines	408182955.00	100968	4042.6962502971238412
8	Rhône	356730381.68	91457	3900.5257299058573975
9	Corse-du-Sud	54674522.00	14529	3763.1304287975772593
10	Gironde	253821608.82	69821	3635.3190131908738059

Total rows: 10 of 10 Query complete 00:00:00.277 Ln 74, Col 1



4. . Les 10 départements où le prix du mètre carré est le plus élevé

- Version 2 chaque bien à poids équivalent – VUE GLOBALE Prix/m2

Version 2 :

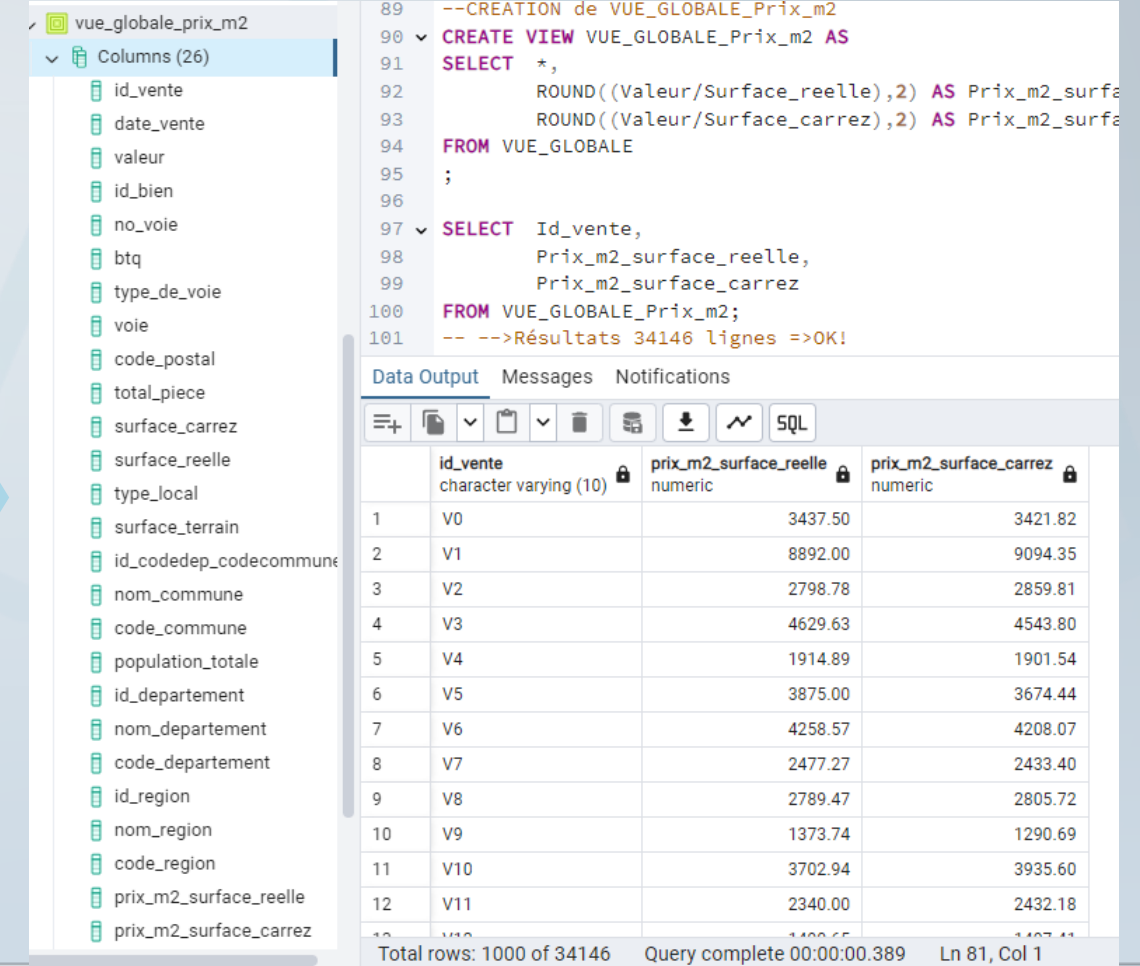
Calculer les Prix/m2 par bien, puis faire une moyenne des Prix/m2 des biens
Chaque bien a un poids équivalent.
C'est le choix que nous conserverons par la suite

```
--CREATION de VUE_GLOBALE_Prix_m2
CREATE VIEW VUE_GLOBALE_Prix_m2 AS
SELECT      *,
            ROUND((Valeur/Surface_reelle),2) AS Prix_m2_surface_reelle,
            ROUND((Valeur/Surface_carrez),2) AS Prix_m2_surface_carrez

FROM VUE_GLOBALE;

SELECT      Id_vente,
            Prix_m2_surface_reelle,
            Prix_m2_surface_carrez

FROM VUE_GLOBALE_Prix_m2;
```



The screenshot displays a database management tool interface. On the left, a tree view shows a schema named 'vue_globale_prix_m2' with 26 columns. A large blue arrow points from the SQL code block on the left towards this interface. The main area on the right shows the SQL editor with the following code:

```
89 --CREATION de VUE_GLOBALE_Prix_m2
90 CREATE VIEW VUE_GLOBALE_Prix_m2 AS
91 SELECT      *,
92             ROUND((Valeur/Surface_reelle),2) AS Prix_m2_surface_reelle,
93             ROUND((Valeur/Surface_carrez),2) AS Prix_m2_surface_carrez
94 FROM VUE_GLOBALE
95 ;
96
97 SELECT      Id_vente,
98             Prix_m2_surface_reelle,
99             Prix_m2_surface_carrez
100 FROM VUE_GLOBALE_Prix_m2;
101 -- -->Résultats 34146 lignes =>OK!
```

Below the editor, the 'Data Output' tab is active, showing a table with the following data:

	id_vente	prix_m2_surface_reelle	prix_m2_surface_carrez
1	V0	3437.50	3421.82
2	V1	8892.00	9094.35
3	V2	2798.78	2859.81
4	V3	4629.63	4543.80
5	V4	1914.89	1901.54
6	V5	3875.00	3674.44
7	V6	4258.57	4208.07
8	V7	2477.27	2433.40
9	V8	2789.47	2805.72
10	V9	1373.74	1290.69
11	V10	3702.94	3935.60
12	V11	2340.00	2432.18

At the bottom, a status bar indicates: 'Total rows: 1000 of 34146', 'Query complete 00:00:00.389', and 'Ln 81, Col 1'.

4. . Les 10 départements où le prix du mètre carré est le plus élevé - Version 2 chaque bien à poids équivalent

Version 2 :

Calculer les Prix/m2 par bien, puis faire une moyenne des Prix/m2 des biens
Chaque bien a un poids équivalent.
C'est le choix que nous conserverons par la suite

```
SELECT      nom_departement AS "Département",  
            ROUND(AVG(Prix_m2_surface_reelle),2) AS "Prix moyen du m2 -S.  
réelle",  
            ROUND(AVG(Prix_m2_surface_carrez),2) AS "Prix moyen du m2 -S.  
carrez"  
  
FROM VUE_GLOBALE_Prix_m2  
GROUP BY nom_departement  
ORDER BY "Prix moyen du m2 -S. réelle" DESC  
LIMIT 10;
```

	Département character varying (100) 🔒	Prix moyen du m2 -S. réelle numeric 🔒	Prix moyen du m2 -S. carrez numeric 🔒
1	Paris	12129.39	12052.82
2	Hauts-de-Seine	7415.28	7219.39
3	Val-de-Marne	5310.67	5336.60
4	Alpes-Maritimes	4685.25	4700.26
5	Seine-Saint-Denis	4371.14	4344.78
6	Haute-Savoie	4149.56	4667.13
7	Yvelines	4126.19	4225.25
8	Rhône	4063.83	4059.28
9	Corse-du-Sud	3921.65	4026.97
10	Gironde	3838.74	3764.14

Total rows: 10 of 10 Query complete 00:00:00.230 Ln 106, Col 1



5. Moyenne du prix au mètre carré d'une maison en Île-de-France.

--PRE-REQUETE 5 : Analyse par région :

```
SELECT Nom_region AS "Région",
       ROUND(AVG(Prix_m2_surface_reelle),2) AS "Prix moyen du m2
-S. réelle pour une Maison",
       ROUND(AVG(Prix_m2_surface_carrez),2) AS "Prix moyen du m2
-S. carrez pour une Maison"
FROM VUE_GLOBALE_Prix_m2
WHERE Type_local = 'Maison'
--AND Nom_region = 'Ile-de-France'
GROUP BY Nom_region
ORDER BY "Prix moyen du m2
-S. réelle pour une Maison" DESC;
```

```
SELECT      Nom_region AS "Région",
            ROUND(AVG(Prix_m2_surface_reelle),2) AS "Prix moyen du m2 -
S. réelle pour une Maison",
            ROUND(AVG(Prix_m2_surface_carrez),2) AS "Prix moyen du m2 -
S. carrez pour une Maison"
FROM VUE_GLOBALE_Prix_m2
WHERE Type_local = 'Maison'
AND Nom_region = 'Ile-de-France'
GROUP BY Nom_region;
```

	Région character varying (100)	Prix moyen du m2 -S. réelle pour une Maison numeric	Prix moyen du m2 -S. carrez pour une Maison numeric
1	Corse	4860.40	5012.81
2	Provence-Alpes-Côte d'Azur	4308.79	4082.80
3	Ile-de-France	3997.71	3745.09
4	Nouvelle-Aquitaine	3294.87	3212.76
5	Martinique	3291.32	2939.35
6	Guyane	3219.14	2858.80
7	Auvergne-Rhône-Alpes	3190.29	3023.11
8	Occitanie	3133.91	3010.93
9	Pays de la Loire	2837.60	2778.73
10	La Réunion	2353.95	2046.79
11	Bretagne	2319.88	2283.50
12	Normandie	2305.76	2238.40
13	Hauts-de-France	2175.68	2514.60
14	Centre-Val de Loire	1904.94	1993.30
15	Grand Est	1810.31	1808.31
16	Bourgogne-Franche-Comté	1404.33	1472.68
Total rows: 16 of 16 Query complete 00:00:00.301 Ln 120, Col 1			

	Région character varying (100)	Prix moyen du m2 -S. réelle pour une Maison numeric	Prix moyen du m2 -S. carrez pour une Maison numeric
1	Ile-de-France	3997.71	3745.09
Total rows: 1 of 1 Query complete 00:00:00.129 Ln 132, Col 1			



6. Les 10 appartements les plus chers avec région et nbre de m2

```
SELECT      Id_bien AS Bien,  
            Type_local,  
            Valeur,  
            Surface_carrez AS "Surface carrez-m2",  
            Surface_reelle AS "Surface réelle-m2",  
            Nom_region AS Région,  
            Prix_m2_surface_carrez AS "Prix/m2 -S. carrez"  
  
FROM VUE_GLOBALE_Prix_m2  
WHERE Type_local = 'Appartement'  
ORDER BY Valeur DESC  
LIMIT 10;
```



	bien character varying	type_local character varying	valeur numeric (11,2)	Surface carrez-m2 numeric (7,2)	Surface réelle-m2 smallint	région character varying	Prix/m2 -S. carrez numeric
1	B30602	Appartement	9000000.00	9.10	10	Ile-de-France	989010.99
2	B5260	Appartement	8600000.00	64.00	62	Ile-de-France	134375.00
3	B3624	Appartement	8577713.00	20.55	289	Ile-de-France	417406.96
4	B7601	Appartement	7620000.00	42.77	42	Ile-de-France	178162.26
5	B9987	Appartement	7600000.00	253.30	200	Ile-de-France	30003.95
6	B17822	Appartement	7535000.00	139.90	143	Ile-de-France	53859.90
7	B409	Appartement	7420000.00	360.95	357	Ile-de-France	20556.86
8	B16356	Appartement	7200000.00	595.00	241	Ile-de-France	12100.84
9	B1923	Appartement	7050000.00	122.56	310	Ile-de-France	57522.85
10	B19160	Appartement	6600000.00	79.38	76	Ile-de-France	83144.37
Total rows: 10 of 10 Query complete 00:00:00.166 Ln 151, Col 1							



7. Taux d'évolution du nbre de ventes entre 1^{er} et 2nd trim. 2020.

```
CREATE VIEW Vente_T1 AS
SELECT COUNT(DISTINCT Id_vente) AS "Nbre Vente T1«
FROM VUE_GLOBALE
WHERE '01/01/2020' <= Date_vente
AND Date_vente < '01/04/2020';
SELECT * FROM Vente_T1 ;
```

```
CREATE VIEW Vente_T2 AS
SELECT COUNT(DISTINCT Id_vente) AS "Nbre Vente T2«
FROM VUE_GLOBALE
WHERE '01/04/2020' <= Date_vente
AND Date_vente < '01/07/2020';
SELECT * FROM Vente_T2 ;
```

```
--REQUETE 7 : Taux d'évolution nbre de ventes entre le 1er et 2nd trim 2020.
SELECT      *,
            ROUND(ROUND((("Nbre Vente T2"- "Nbre Vente T1")*100),2)
                /"Nbre Vente T1",2) AS "Taux d'évolution du Nbre
de Vente de T1 à T2(%) "

FROM Vente_T1
CROSS JOIN Vente_T2;
```



	Nbre Vente T1 bigint
1	16768



	Nbre Vente T2 bigint
1	17378

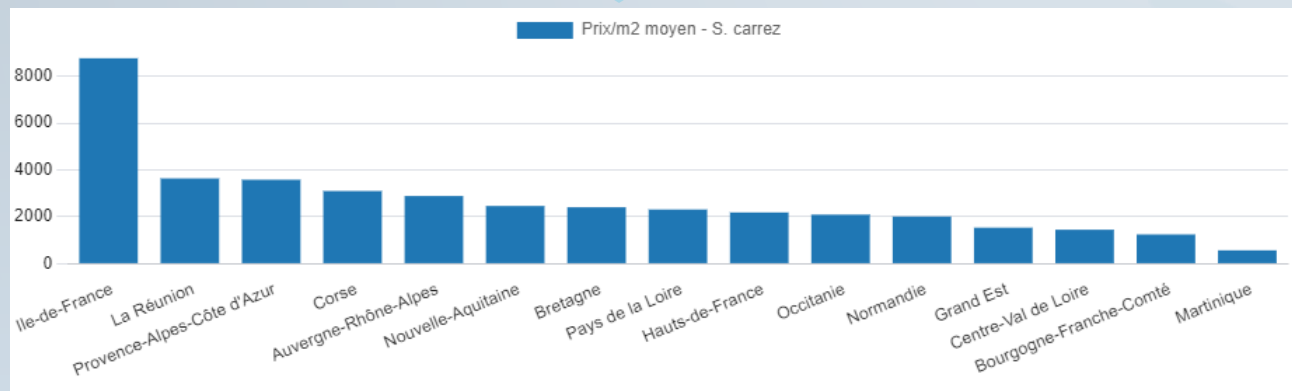


	Nbre Vente T1 bigint	Nbre Vente T2 bigint	Taux d'évolution du Nbre de Vente de T1 à T2(%) numeric
1	16768	17378	3.64



8. Classement des régions par Prix/m2 des appartement de + de 4 pièces.

```
SELECT      nom_region AS "Région",  
            ROUND(AVG(prix_m2_surface_carrez),2) AS "Prix/m2 moyen - S. carrez "  
  
FROM VUE_GLOBALE_PRIX_M2  
WHERE type_local = 'Appartement'  
AND total_piece > 4  
GROUP BY nom_region  
ORDER BY "Prix/m2 moyen - S. carrez" DESC;
```



	Région character varying (100)	Prix/m2 moyen - S. carrez numeric
1	Ile-de-France	8770.44
2	La Réunion	3641.82
3	Provence-Alpes-Côte d'Azur	3587.65
4	Corse	3104.88
5	Auvergne-Rhône-Alpes	2891.38
6	Nouvelle-Aquitaine	2465.48
7	Bretagne	2412.05
8	Pays de la Loire	2315.76
9	Hauts-de-France	2189.93
10	Occitanie	2097.23
11	Normandie	2015.77
12	Grand Est	1540.89
13	Centre-Val de Loire	1453.11
14	Bourgogne-Franche-Comté	1251.19
15	Martinique	573.48

9. Liste des communes avec au moins 50 ventes au 1er trim. 2020

```
SELECT      nom_commune,  
            COUNT(Id_vente) AS "Nbre de Ventes"  
  
FROM VUE_GLOBALE  
WHERE date_vente BETWEEN '01/01/2020' AND '31/03/2020'  
GROUP BY nom_commune  
HAVING COUNT(Id_vente)>=50  
ORDER BY "Nbre de Ventes" DESC;
```



	nom_commune character varying (100)	Nbre de Ventes bigint
1	Paris 17e Arrondissement	228
2	Paris 15e Arrondissement	215
3	Paris 18e Arrondissement	209
4	Nice	173
5	Paris 11e Arrondissement	169
6	Paris 16e Arrondissement	165
7	Bordeaux	157
8	Paris 14e Arrondissement	146
9	Paris 20e Arrondissement	127
10	Nantes	119
11	Paris 19e Arrondissement	116
12	Paris 12e Arrondissement	110
13	Paris 10e Arrondissement	109
14	Paris 9e Arrondissement	106
15	Grenoble	106
16	Boulogne-Billancourt	99
17	Paris 13e Arrondissement	94
18	Paris 7e Arrondissement	87
19	Paris 6e Arrondissement	86
20	Marseille 8e Arrondissement	81
Total rows: 48 of 48 Query complete 00:00:00.319		



10. Différence(en %) du Prix/m2 entre un appart. de 2pcs et de 3pcs 1/2

Exploration :

Analyse du Prix/m2 (S. carrez) par nbre de pièces

```
SELECT      total_piece,  
            COUNT(Id_vente) AS "Nbre de vente",  
            ROUND(AVG(prix_m2_surface_carrez),2) AS "Prix/m2 moyen  
- S. carrez "  
  
FROM VUE_GLOBALE_PRIX_M2  
WHERE type_local = 'Appartement'  
GROUP BY total_piece  
--HAVING total_piece IN (2,3)  
ORDER BY total_piece DESC;
```

	total_piece smallint	Nbre de vente bigint	Prix/m2 moyen - S. carrez numeric
1	11	1	2730.31
2	10	2	8936.04
3	9	8	6966.27
4	8	17	11106.85
5	7	54	17917.90
6	6	203	6493.11
7	5	1114	4705.04
8	4	4458	4101.48
9	3	8964	4300.80
10	2	9773	4908.57
11	1	6734	5982.35
12	0	29	5407.92
Total rows: 12 of 12 Query complete 00:00:00.335 Ln 265,			



10. Différence(en %) du Prix/m2 entre un appart. de 2pcs et de 3pcs 1/2

```
CREATE VIEW APP_2P_PMM2 AS
SELECT      ROUND(AVG(prix_m2_surface_carrez),2) AS "Prix/m2 moyen - S. carrez-App 2pièces "
FROM VUE_GLOBALE_PRIX_M2
WHERE type_local = 'Appartement'
GROUP BY total_piece
HAVING total_piece =2
ORDER BY total_piece DESC;
SELECT * FROM APP_2P_PMM2 ;
```

```
CREATE VIEW APP_3P_PMM2 AS
SELECT      ROUND(AVG(prix_m2_surface_carrez),2) AS "Prix/m2 moyen - S. carrez-App 3pièces "
FROM VUE_GLOBALE_PRIX_M2
WHERE type_local = 'Appartement'
GROUP BY total_piece
HAVING total_piece =3
ORDER BY total_piece DESC;
SELECT * FROM APP_3P_PMM2 ;
```

```
SELECT      *,
ROUND(("Prix/m2 moyen - S. carrez-App 3pièces" - "Prix/m2 moyen - S. carrez-App 2pièces")*100
      /"Prix/m2 moyen - S. carrez-App 2pièces",2) AS "Différence de Prix/m2 moyen - S.
carrez-App 3pièces VS 2pièces (en %)"
FROM APP_2P_PMM2
CROSS JOIN APP_3P_PMM2;
```

	Prix/m2 moyen - S. carrez-App 2pièces numeric
1	4908.57

	Prix/m2 moyen - S. carrez-App 3pièces numeric
1	4300.80

Prix/m2 moyen - S. carrez-App 2pièces numeric	Prix/m2 moyen - S. carrez-App 3pièces numeric
4908.57	4300.80

Différence de Prix/m2 moyen - S. carrez-App 3pièces VS 2pièces numeric
-12.38



11. Moyennes des Valeurs foncières pour le top 3 des communes des départements 6, 13, 33, 59 et 69. –CREATION de VUE par DEPARTEMENT

```
CREATE VIEW VF_D006 AS
SELECT      id_departement AS Departement,
            nom_commune AS Commune,
            ROUND(AVG(Valeur)) AS Valeur_fonciere_moyenne
FROM VUE_GLOBALE
WHERE id_departement = 'D006'
GROUP BY id_departement , nom_commune
ORDER BY AVG(Valeur) DESC
LIMIT 3;
```

```
CREATE VIEW VF_D013 AS
SELECT      id_departement AS Departement,
            nom_commune AS Commune,
            ROUND(AVG(Valeur)) AS Valeur_fonciere_moyenne
FROM VUE_GLOBALE
WHERE id_departement = 'D013'
GROUP BY id_departement , nom_commune
ORDER BY AVG(Valeur) DESC
LIMIT 3;
```

```
CREATE VIEW VF_D033 AS
[...]
```

```
WHERE id_departement = 'D033'
```

```
[...]
```



	departement character (4) 🔒	commune character varying (100) 🔒	valeur_fonciere_moyenne numeric 🔒
1	D006	Saint-Jean-Cap-Ferrat	968750
2	D006	Eze	655000
3	D006	Mouans-Sartoux	476898

	departement character (4) 🔒	commune character varying (100) 🔒	valeur_fonciere_moyenne numeric 🔒
1	D013	Gignac-la-Nerthe	330000
2	D013	Saint-Savournin	314425
3	D013	Cassis	313417

	departement character (4) 🔒	commune character varying (100) 🔒	valeur_fonciere_moyenne numeric 🔒
1	D033	Lège-Cap-Ferret	549501
2	D033	Vayres	335000
3	D033	Arcachon	307436

	departement character (4) 🔒	commune character varying (100) 🔒	valeur_fonciere_moyenne numeric 🔒
1	D059	Bersée	433202
2	D059	Cysoing	408550
3	D059	Halluin	322250

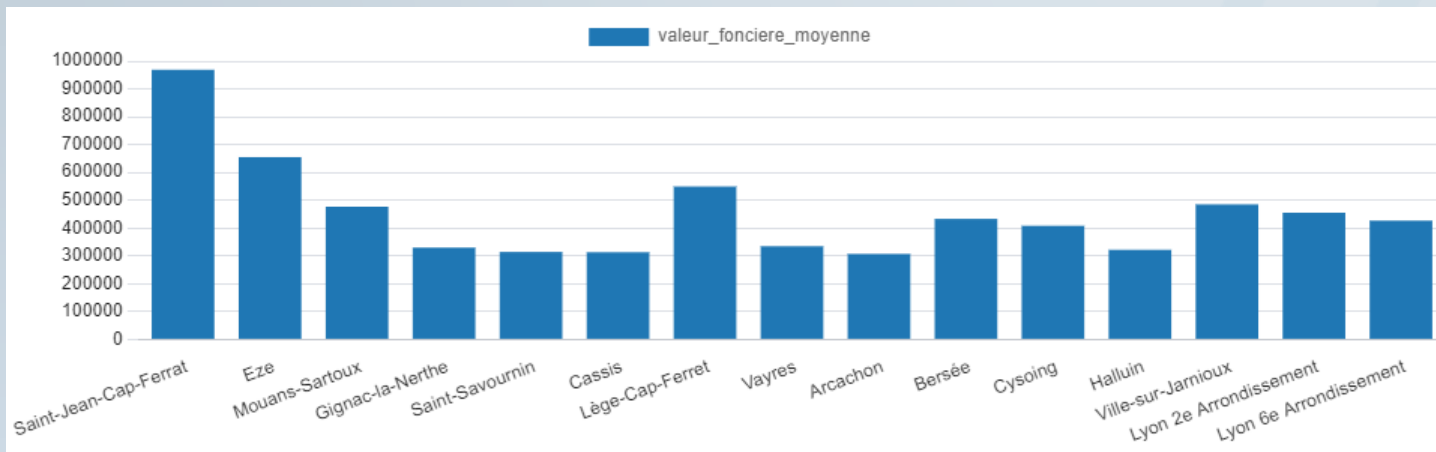
	departement character (4) 🔒	commune character varying (100) 🔒	valeur_fonciere_moyenne numeric 🔒
1	D069	Ville-sur-Jarnioux	485300
2	D069	Lyon 2e Arrondissement	455217
3	D069	Lyon 6e Arrondissement	426968



11. Moyennes des Valeurs foncières pour le top 3 des communes des départements 6, 13, 33, 59 et 69.

--REUNION DES REQUETES :

```
SELECT * FROM VF_D006
UNION
SELECT * FROM VF_D013
UNION
SELECT * FROM VF_D033
UNION
SELECT * FROM VF_D059
UNION
SELECT * FROM VF_D069
ORDER BY Departement , Valeur_fonciere_moyenne DESC;
```



	departement character (4)	commune character varying (100)	valeur_fonciere_moyenne numeric
1	D006	Saint-Jean-Cap-Ferrat	968750
2	D006	Eze	655000
3	D006	Mouans-Sartoux	476898
4	D013	Gignac-la-Nerthe	330000
5	D013	Saint-Savournin	314425
6	D013	Cassis	313417
7	D033	Lège-Cap-Ferret	549501
8	D033	Vayres	335000
9	D033	Arcachon	307436
10	D059	Bersée	433202
11	D059	Cysoing	408550
12	D059	Halluin	322250
13	D069	Ville-sur-Jarnioux	485300
14	D069	Lyon 2e Arrondissement	455217
15	D069	Lyon 6e Arrondissement	426968

12. Les 20 communes avec le plus de transactions /1000 habitants pour les communes de plus de 10 000 habitants.

```
SELECT      id_departement AS Departement,
            nom_commune AS Commune,
            ROUND( ROUND(( COUNT(Id_Vente)*1000 ),2) /population_totale ,2) AS "Nbre
de Ventes/Milliers d'hab",
            population_totale AS population,
            COUNT(Id_Vente) AS "Nombre de Ventes "

FROM VUE_GLOBALE
WHERE population_totale >10000
GROUP BY id_codedep_codecommune, nom_commune, id_departement, population_totale
ORDER BY "Nbre de Ventes/Milliers d'hab" DESC
LIMIT 20;
```

	departement character (4)	commune character varying (100)	Nbre de Ventes /Milliers d'hab numeric	population integer	Nombre de Ventes bigint
1	D075	Paris 2e Arrondissement	5.84	21735	127
2	D075	Paris 1er Arrondissement	4.86	16055	78
3	D075	Paris 3e Arrondissement	4.69	34306	161
4	D033	Arcachon	4.62	11898	55
5	D044	La Baule-Escoublac	4.58	16797	77
6	D075	Paris 4e Arrondissement	4.05	29390	119
7	D006	Roquebrune-Cap-Martin	3.99	13041	52
8	D075	Paris 8e Arrondissement	3.83	36250	139
9	D083	Sanary-sur-Mer	3.50	17160	60
10	D075	Paris 9e Arrondissement	3.43	60563	208
11	D083	La Londe-les-Maures	3.43	10776	37
12	D075	Paris 6e Arrondissement	3.38	41171	139
13	D083	Saint-Cyr-sur-Mer	3.16	11725	37
14	D060	Chantilly	3.13	11178	35
15	D094	Saint-Mandé	3.06	22576	69
16	D044	Pornichet	3.06	11440	35
17	D075	Paris 10e Arrondissement	3.04	86863	264
18	D006	Menton	2.94	30981	91
19	D085	Saint-Hilaire-de-Riez	2.87	11501	33
20	D094	Vincennes	2.81	50230	141

Merci !

Merci !



Laplace Immo

ANNEXE



3NF respectée

MLD :

REGION = (ID_REGION CHAR(3), NOM_REGION VARCHAR(100), CODE_REGION VARCHAR(2));

DEPARTEMENT = (ID_DEPARTEMENT CHAR(4), NOM_DEPARTEMENT VARCHAR(100), CODE_DEPARTEMENT VARCHAR(3), #ID_REGION);

COMMUNE = (ID_CODEDEP_CODECOMMUNE CHAR(6), NOM_COMMUNE VARCHAR(100), CODE_COMMUNE CHAR(3), POPULATION_TOTALE INT, #ID_DEPARTEMENT);

BIEN = (ID_BIEN VARCHAR(10), NO_VOIE SMALLINT, BTQ CHAR(1), TYPE_DE_VOIE VARCHAR(10), VOIE VARCHAR(60), CODE_POSTAL INT, TOTAL_PIECE SMALLINT, SURFACE_CARREZ DECIMAL(7,2), SURFACE_REELLE SMALLINT, TYPE_LOCAL VARCHAR(20), SURFACE_TERRAIN SMALLINT, #ID_CODEDEP_CODECOMMUNE);

VENTE = (ID_VENTE VARCHAR(10), DATE_VENTE DATE, VALEUR DECIMAL(11,2), #ID_BIEN);

- 1NF respectée : tout attribut est atomique
- 2NF respectée : ...+tout attribut non clé ne dépend pas d'une partie de clé
- 3NF respectée : ...+tout attribut n'appartenant pas à une clé ne dépend pas d'un autre attribut non clé



Nettoyage sur le fichier Valeurs foncières : AJOUT I code postal : pas de chgmt nbr de lignes

	Code postal	Code departement	Code commune	Code ID commune
908	20090.0	2A	4	618
909	20000.0	2A	4	616
26834	NaN	2A	4	658

Il semble qu'il y ait une erreur sur l'index 26834 : codepostal = 20090.0 et code ID commune=618; car c'est la seule valeur ainsi.

NETTOYONS le fichier valeurs_foncières_nettoyé_1 avec cette information

=>**CORRECTION : REMPLISSONS LE CODE POSTAL ABSENT**

```
valeurs_foncières_nettoyé_1.loc[(valeurs_foncières_nettoyé_1['Code postal']).isna(),'Code postal']=20090.0
```



Nettoyage sur le fichier Valeurs foncières : SUPPRESSIONS de 18 lignes qui n'ont pas de valeurs foncières (NAN) : =>34151 lignes

```
[94]: (valeurs_foncieres_nettoye_1.loc[(valeurs_foncieres_nettoye_1['Valeur fonciere']).isna(), :]).index
[94]: Index([ 1676,  3620,  3918,  5518,  8280, 11345, 11516, 18482, 19394, 26724,
          27910, 28962, 29552, 29722, 31141, 31699, 32503, 34010],
          dtype='int64')
[95]: liste_1 = (valeurs_foncieres_nettoye_1.loc[(valeurs_foncieres_nettoye_1['Valeur fonciere']).isna(), :]).index
valeurs_foncieres_nettoye_1.drop(index=liste_1, inplace=True)
```

```
[96]: Description_Fichier_Tableau(pd.DataFrame(valeurs_foncieres_nettoye_1['Valeur fonciere']))
```

```
[96]:
```

	Type	Nb lignes	Valeurs non-vides	Valeurs vides	Valeurs distinctes
Valeur fonciere	float64	34151	34151	0	9680

Nettoyage sur le fichier Valeurs foncières : SUPPRESSIONS de 18 lignes qui n'ont pas de valeurs foncières (NaN) : =>34151 lignes

[167]:

```
valeurs_foncieres_nettoye_1.loc[(valeurs_foncieres_nettoye_1['Valeur_fonciere']).isna(), liste_col_utiles_VF]
```

[167]:

	No voie	Type de voie	Voie	B/T/Q	Code postal	Surface Carrez du 1er lot	Type local	Surface reelle bati	Nombre pieces principales	Surface terrain	Code departement	Code commune	Date mutation	Valeur fonciere
1676	194.0	RUE	DE RIVOLI	B	75001.0	197.92	Appartement	225	6	NaN	75	101	2020-01-13	NaN
3620	23.0	RUE	STE CROIX BRETONNERIE	NaN	75004.0	21.00	Appartement	23	1	NaN	75	104	2020-01-22	NaN
3918	15.0	QUAI	DE LA SOMME	NaN	76200.0	34.64	Appartement	42	2	NaN	76	217	2020-01-23	NaN
5518	13.0	RUE	DE LA PORTE NEUVE	NaN	62200.0	40.02	Appartement	43	2	NaN	62	160	2020-01-30	NaN
8280	5360.0	ESP	DESPIERRE	NaN	5560.0	17.34	Appartement	17	1	NaN	05	177	2020-02-12	NaN
11345	74.0	RUE	ARTHUR LAMENDIN	NaN	62400.0	35.47	Appartement	37	2	NaN	62	119	2020-02-26	NaN
11516	12.0	AV	PABLO PICASSO	NaN	93420.0	46.95	Appartement	63	2	NaN	93	78	2020-02-26	NaN
18482	3.0	RUE	DANTON	NaN	84800.0	72.00	Appartement	91	4	NaN	84	54	2020-04-24	NaN
19394	25.0	RUE	DES PARAMIDEAUX	NaN	6110.0	52.02	Appartement	56	2	NaN	06	30	2020-05-04	NaN
26724	5421.0	RES	DU PARC 3	NaN	20167.0	45.64	Appartement	46	2	NaN	2A	271	2020-05-30	NaN
27910	155.0	RUE	DES FAUVETTES	NaN	16600.0	85.22	Maison	82	5	NaN	16	291	2020-06-05	NaN
28962	2.0	RUE	DE LA MUETTE	NaN	94130.0	31.61	Appartement	32	1	NaN	94	52	2020-06-09	NaN
29552	8.0	AV	BERLIOZ	NaN	93270.0	79.63	Appartement	80	4	NaN	93	71	2020-06-11	NaN
29722	7.0	RUE	ALFRED SISLEY	NaN	44800.0	93.92	Maison	94	5	NaN	44	162	2020-06-12	NaN
31141	15.0	RUE	NEUVE POPINCOURT	NaN	75011.0	69.00	Appartement	41	2	NaN	75	111	2020-06-18	NaN
31699	30.0	RUE	CHAUVEAU LAGARDE	NaN	28000.0	26.81	Appartement	39	2	NaN	28	85	2020-06-20	NaN
32503	4.0	BD	MICHELET	NaN	19100.0	40.62	Appartement	40	2	NaN	19	31	2020-06-25	NaN
34010	9200.0	AV	DE LATTRE DE TASSIGNY	NaN	83270.0	32.64	Appartement	32	2	NaN	83	112	2020-06-30	NaN

```
Code departement
75    3
62    2
93    2
76    1
05    1
84    1
06    1
2A    1
16    1
94    1
44    1
28    1
19    1
83    1
Name: count, dtype: int64
```

Nettoyage sur le fichier Valeurs foncières : SUPPRESSION de 2 lignes avec des surfaces réelles bâties $\leq 4\text{m}^2$ $\Rightarrow$ 34149 lignes

```
b. ETUDE des Faibles surfaces baties
```

```
[110]: liste_attributs_surface_2 = ['Surface Carrez du 1er lot', 'Surface reelle bati',  
                                   'Nombre pieces principales', 'Surface terrain',  
                                   'Valeur fonciere', 'Type local']  
(valeurs_foncieres_nettoye_1.loc[ valeurs_foncieres_nettoye_1['Surface reelle bati'] <= 4, : ])[liste_attributs_surface_2]
```

```
[110]:
```

	Surface Carrez du 1er lot	Surface reelle bati	Nombre pieces principales	Surface terrain	Valeur fonciere	Type local
22380	2.36	2	0	NaN	116500.0	Appartement
30429	9.62	1	1	NaN	175000.0	Appartement

=>CORRECTION : SUPPRIMONS les 2 lignes avec des surfaces réelles bâties $\leq 4\text{m}^2$

```
[113]: valeurs_foncieres_nettoye_1.shape
```

```
[113]: (34151, 45)
```

```
[114]: df_4 = (valeurs_foncieres_nettoye_1.loc[ valeurs_foncieres_nettoye_1['Surface reelle bati'] <= 4, : ])[liste_attributs_surface_2]  
liste_index_aberrant_2 = df_4.index  
  
valeurs_foncieres_nettoye_1.drop(index=liste_index_aberrant_2, inplace=True )
```

```
[115]: valeurs_foncieres_nettoye_1.shape
```

```
[115]: (34149, 45)
```


Nettoyage sur le fichier Valeurs foncières :

SUPPRESSION de 3 lignes avec des valeurs foncières $\leq 2_500$ euros : $\Rightarrow 34146$ lignes

```
[127]: liste_attributs_surface_2 = ['Surface Carrez du 1er lot', 'Surface reelle bati',  
                                'Nombre pieces principales', 'Surface terrain',  
                                'Valeur fonciere', 'Type local']  
(valeurs_foncieres_nettoye_1.loc[ valeurs_foncieres_nettoye_1['Valeur fonciere'] <= 2_500, :]) [liste_attributs_surface_2]
```

```
[127]:
```

	Surface Carrez du 1er lot	Surface reelle bati	Nombre pieces principales	Surface terrain	Valeur fonciere	Type local
5472	23.24	23	1	NaN	1021.05	Appartement
17234	2.10	42	3	NaN	537.50	Appartement
20576	15.00	57	3	NaN	2200.00	Appartement

```
[130]: valeurs_foncieres_nettoye_1.shape
```

```
[130]: (34149, 45)
```

```
[131]: df_5 = valeurs_foncieres_nettoye_1.loc[ valeurs_foncieres_nettoye_1['Valeur fonciere'] <= 2_500, :]  
liste_index_aberrant_3 = df_5.index
```

```
[132]: valeurs_foncieres_nettoye_1.shape
```

```
[132]: (34146, 45)
```

8. Le classement des régions par rapport au prix au mètre carré des appartement de plus de 4 pièces. Complément

```
--Nbre de ventes pour 'Guadeloupe','Guyane'  
SELECT      nom_region AS Region,  
            type_local,  
            total_piece,  
            COUNT(DISTINCT Id_vente) AS "Nbre de Vente "  
FROM VUE_GLOBALE  
WHERE nom_region IN ('Guadeloupe','Guyane')  
GROUP BY nom_region, type_local, total_piece  
ORDER BY nom_region, type_local, total_piece DESC;
```



	region character varying (100) 🔒	type_local character varying (20) 🔒	total_piece smallint 🔒	Nbre de Vente bigint 🔒
1	Guadeloupe	Appartement	1	2
2	Guyane	Appartement	4	2
3	Guyane	Appartement	3	9
4	Guyane	Appartement	2	17
5	Guyane	Appartement	1	6
6	Guyane	Maison	4	3
7	Guyane	Maison	3	2

La Guadeloupe et la Guyane n'ont pas vendu d'appartement de plus de 4 pièces.



ANNEXE - FIN



